

Week 6 Lab: Code Review & RTOS Foundations

EE0827 Embedded System Applications

Overview

This lab has two parts. Part 1 reviews the bare-metal HAL code patterns from Weeks 2–5 and identifies their limitations. Part 2 introduces FreeRTOS by converting a monolithic super loop into independent tasks.

Time allocation: 90 minutes

- Bare-metal code review & decomposition: 35 min
- FreeRTOS multi-task project: 45 min
- Cleanup: 10 min

Learning Objectives

1. Identify the common HAL peripheral pattern: clock → configure → init → start
2. Read and explain timer, ADC, PWM, and UART configuration code
3. Recognize super loop limitations in a combined application
4. Create a FreeRTOS project with multiple concurrent tasks
5. Compare `osDelay()` vs `HAL_Delay()` behavior

The HAL Pattern

Every peripheral follows the same structure. Fill in the blanks:

```
// 1. Enable clock
__HAL_RCC_GPIOx_CLK_ENABLE();
__HAL_RCC_TIMx_CLK_ENABLE();
__HAL_RCC_ADC1_CLK_ENABLE();

// 2. Configure struct
GPIO_InitStruct.Pin   = ____;
GPIO_InitStruct.Mode  = ____;

// 3. Call HAL init
HAL_GPIO_Init(GPIOx, &GPIO_InitStruct);

// 4. Start peripheral (if needed)
HAL_TIM_PWM_Start(&htimX, TIM_CHANNEL_X);
HAL_ADC_Start_IT(&hadc1);
```

Question: Which step does CubeMX generate automatically?

Peripheral Review

For each peripheral, write the key HAL function(s) used in previous labs:

Peripheral	Init Function	Runtime Function	Week
GPIO Output	MX_GPIO_Init()	HAL_GPIO_TogglePin()	W02
ADC			W03
Timer/PWM			W04
UART			W03
EXTI (interrupt)			W03

Timer Configuration

Given this timer setup from W04:

```
htim3.Init.Prescaler    = 83;      // 84MHz / (83+1) = 1MHz  
htim3.Init.Period       = 999;     // 1MHz / (999+1) = 1kHz PWM  
htim3.Init.CounterMode = TIM_COUNTERMODE_UP;
```

1. What is the timer tick frequency? _____
2. What is the PWM output frequency? _____
3. If `__HAL_TIM_SET_COMPARE(&htim3, TIM_CHANNEL_1, 500)`, what is the duty cycle?

ADC Configuration

From W03/W05:

```
hadc1.Init.Resolution    = ADC_RESOLUTION_12B;  
hadc1.Init.ScanConvMode  = DISABLE;  
hadc1.Init.ContinuousConvMode = DISABLE;  
hadc1.Init.ExternalTrigConv = ADC_SOFTWARE_START;
```

1. What is the ADC value range with 12-bit resolution? _____
2. If $V_{ref} = 3.3V$ and ADC reads 2048, what is the input voltage? _____
3. What does `ContinuousConvMode = DISABLE` mean? _____

Super Loop Decomposition

Below is a combined bare-metal program. Read it and answer the questions.

```
int main(void) {
    HAL_Init();
    SystemClock_Config();
    MX_GPIO_Init();
    MX_ADC1_Init();
    MX_TIM3_Init();
    MX_USART2_UART_Init();

    HAL_TIM_PWM_Start(&htim3, TIM_CHANNEL_1);
    HAL_ADC_Start(&hadc1);

    while (1) {
        // Read temperature sensor
        HAL_ADC_PollForConversion(&hadc1, 100);
        uint16_t adc_val = HAL_ADC_GetValue(&hadc1);

        // Simple moving average filter
        static uint16_t buf[8]; static uint8_t idx = 0;
        buf[idx++ & 7] = adc_val;
        uint32_t sum = 0;
        for (int i = 0; i < 8; i++) sum += buf[i];
        uint16_t filtered = sum >> 3;

        // Adjust motor PWM based on temperature
        uint16_t duty = (filtered > 2048) ? 800 : 400;
        __HAL_TIM_SET_COMPARE(&htim3, TIM_CHANNEL_1, duty);

        // Send data over UART
        char msg[64];
        sprintf(msg, "ADC:%d PWM:%d\r\n", filtered, duty);
        HAL_UART_Transmit(&huart2, (uint8_t*)msg, strlen(msg), 100);

        // Check button
        if (HAL_GPIO_ReadPin(BTN_GPIO_Port, BTN_Pin) == GPIO_PIN_RESET)
            HAL_GPIO_TogglePin(LED_GPIO_Port, LED_Pin);

        HAL_Delay(100);
    }
}
```

```
}
```

1. List 4 independent tasks this program performs:

#	Task	Approx. time per iteration
1		
2		
3		
4		

2. What is the worst-case button response time? _____
3. If HAL_UART_Transmit takes 20ms, how does it affect motor control timing?

-
4. Propose RTOS task priorities for the 4 tasks above (High / Normal / Low):

Task	Proposed Priority	Why?
------	-------------------	------

Interrupt-Driven Version

Here is the same program refactored to use interrupts for the button and ADC:

```
volatile uint8_t btn_pressed = 0;
volatile uint16_t adc_val = 0;
volatile uint8_t adc_ready = 0;

// Callbacks (called from ISR context)
void HAL_GPIO_EXTI_Callback(uint16_t GPIO_Pin) {
    if (GPIO_Pin == BTN_Pin)
        btn_pressed = 1;
}

void HAL_ADC_ConvCpltCallback(ADC_HandleTypeDef *hadc) {
```

```

    adc_val = HAL_ADC_GetValue(hadc);
    adc_ready = 1;
}

int main(void) {
    HAL_Init();
    SystemClock_Config();
    MX_GPIO_Init();          // BTN configured as EXTI
    MX_ADC1_Init();
    MX_TIM3_Init();
    MX_USART2_UART_Init();

    HAL_TIM_PWM_Start(&htim3, TIM_CHANNEL_1);
    HAL_ADC_Start_IT(&hadc1); // interrupt mode

    while (1) {
        if (adc_ready) {
            adc_ready = 0;
            // filter + update PWM
            uint16_t duty = (adc_val > 2048) ? 800 : 400;
            __HAL_TIM_SET_COMPARE(&htim3, TIM_CHANNEL_1, duty);
            HAL_ADC_Start_IT(&hadc1); // restart
        }
        if (btn_pressed) {
            btn_pressed = 0;
            HAL_GPIO_TogglePin(LED_GPIO_Port, LED_Pin);
        }
        // UART logging
        char msg[64];
        sprintf(msg, "ADC:%d\r\n", adc_val);
        HAL_UART_Transmit(&huart2, (uint8_t*)msg, strlen(msg), 100);
        HAL_Delay(100);
    }
}

```

1. What improved compared to the polling super loop?

2. What problems remain? (Think about: UART blocking, shared variables, adding new features)

3. Compare all three approaches:

Aspect	Polling Super Loop	Interrupts + Super Loop	RTOS
Button response time			
Can UART block other work?			
Adding a 5th feature			
Shared data safety			

Project Setup

1. Create new STM32 project for **NUCLEO-F401RE**
2. Configure GPIO: PA5 (output, LED), PA0 (output, LED2), PC13 (input, button)
3. Enable **Middleware** → **FREERTOS** → **CMSIS_V2**
4. In Tasks tab, create 3 tasks:

Task Name	Priority	Stack	Entry Function
ButtonTask	osPriorityHigh	128	StartButtonTask
FastLED	osPriorityNormal	128	StartFastLED
Heartbeat	osPriorityLow	128	StartHeartbeat

5. Generate code

Implement Tasks

Open `freertos.c` and replace the generated task bodies:

Heartbeat (1Hz blink on PA5):

```
void StartHeartbeat(void *argument) {
    for (;;) {
        HAL_GPIO_TogglePin(LED_GPIO_Port, LED_Pin);
        osDelay(1000);
    }
}
```

FastLED (variable rate on PA0):

```
extern volatile uint8_t fast_mode;

void StartFastLED(void *argument) {
    for (;;) {
        HAL_GPIO_TogglePin(LED2_GPIO_Port, LED2_Pin);
        osDelay(fast_mode ? 50 : 200);
    }
}
```


ButtonTask (toggle mode on press):

```
volatile uint8_t fast_mode = 0;

void StartButtonTask(void *argument) {
    GPIO_PinState last = GPIO_PIN_SET;
    for (;;) {
        GPIO_PinState now = HAL_GPIO_ReadPin(BTN_GPIO_Port, BTN_Pin);
        if (last == GPIO_PIN_SET && now == GPIO_PIN_RESET) {
            osDelay(50); // debounce
            if (HAL_GPIO_ReadPin(BTN_GPIO_Port, BTN_Pin) == GPIO_PIN_RESET)
                fast_mode = !fast_mode;
        }
        last = now;
        osDelay(10);
    }
}
```

Build, flash, and verify both LEDs blink independently and button toggles the fast LED rate.

HAL_Delay vs osDelay Experiment

Temporarily change the Heartbeat task:

```
void StartHeartbeat(void *argument) {  
    for (;;) {  
        HAL_GPIO_TogglePin(LED_GPIO_Port, LED_Pin);  
        HAL_Delay(1000); // <-- busy wait!  
    }  
}
```

Observation	With osDelay	With HAL_Delay
Heartbeat LED blinks?		
FastLED still blinks?		
Button responsive?		

Why? _____

Revert to osDelay(1000) when done.

Measurement Summary

ID	Measurement	Expected	Measured
M1	Heartbeat period	2s (1Hz toggle)	
M2	FastLED period (normal)	400ms	
M3	FastLED period (fast)	100ms	
M4	Button response	<50ms	

Analysis Questions

1. Why do both LEDs appear to blink simultaneously on a single-core MCU?

2. In Part 1's super loop, UART transmission blocked everything for 20ms. How would you solve this with RTOS?

3. What happens if ButtonTask runs an infinite loop without any `osDelay()`?

Deliverables

- ☐ Part 1 worksheet completed (peripheral review, timer/ADC questions, decomposition)
- ☐ Working 3-task FreeRTOS project
- ☐ HAL_Delay vs osDelay observation table filled
- ☐ Analysis questions answered

Key Takeaways

1. **All HAL peripherals follow the same pattern** — clock, configure, init, start
2. **Super loops can't prioritize** — every task waits for every other task
3. **FreeRTOS tasks run independently** — each with its own timing and priority
4. **osDelay() yields the CPU** — HAL_Delay() blocks everything
5. **Task decomposition is a design decision** — identify independent responsibilities first